

CTT009 – COMPUTATIONAL THINKING

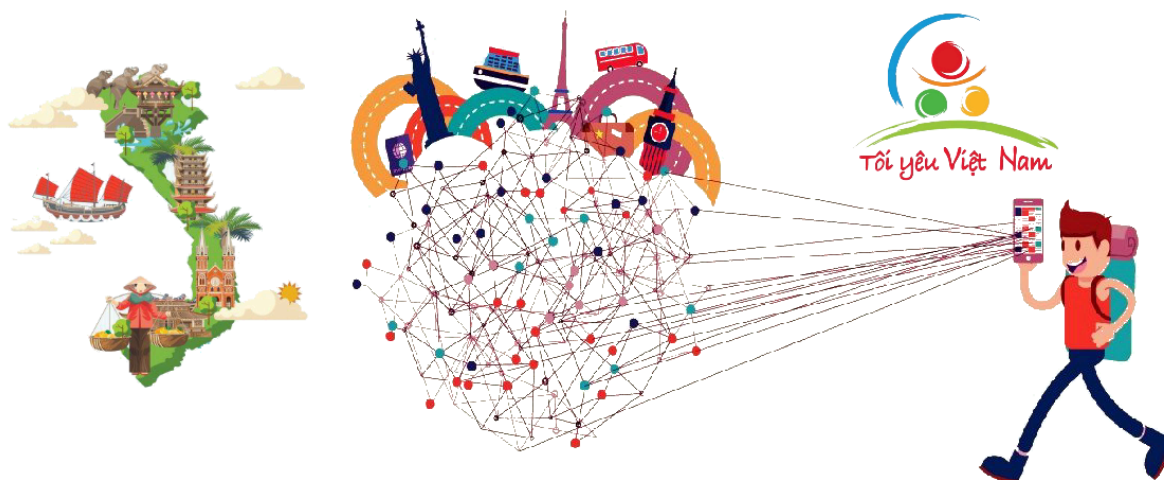
PROJECT: SMART TOURISM SYSTEM

1. Objective

This project aims to help students apply computational thinking steps to solve a real-world problem. Through it, students will not only learn to analyze problems, design and simulate systems but also develop an orientation toward applying artificial intelligence to societal fields. In addition to practicing logical, abstract, and modeling thinking, students will enhance important soft skills such as teamwork, presentation, and academic debate.

2. Description

Tourism is a key economic sector in Vietnam, yet the tourist experience still faces limitations such as rigid itineraries, poor personalization, and lack of intelligent information support. Building an AI-powered smart tourism system can address these issues—for instance, recommending itineraries suited to each traveler's preferences, time, and budget, optimizing costs, and offering a more convenient and engaging experience. This project demonstrates the practical value of computational thinking in designing systems closely connected to real life.



Nguồn ảnh: itourism

Students will propose ideas and simulate a smart tourism system. During implementation, each team can choose specific functions to realize at a simulation level, such as: a travel recommendation chatbot, a restaurant recommendation tool based on budget, landmark recognition from images, or a visual dashboard of travel routes, etc.



Suggested Directions

Transportation (Get in, Travel around)

- *Before the trip*: route optimization for multi-day itineraries using optimization algorithms.
- *During the trip*: suggest transportation modes (bus, taxi, e-bike) based on current location and cost using recommendation systems.
- *Safety*: warn about traffic congestion and busy routes using real-time data analysis.

Sightseeing (Sightsee)

- *Before*: recommend destinations by interest (culture, nature, shopping) using recommendation systems.
- *During*: identify landmarks from images using computer vision and provide information via information retrieval.
- *After*: automatically generate photo albums with location metadata using image recognition + metadata extraction.

Entertainment (Do, Play)

- *Before*: recommend suitable activities by age or group type using recommendation systems.
- *During*: suggest ongoing events and local festivals using NLP + information retrieval; virtual tours using AR/VR.

Shopping (Buy)

- *During*: recommend nearby markets/souvenir shops based on GPS using recommendation systems; identify products from images to find where to buy them using computer vision.
- *After*: analyze shopping receipts to recommend next destinations using data mining.

Food (Eat, Drink)

- *During*: recommend local dishes according to taste and budget using recommendation systems; restaurant chatbots; translate menus with NLP + machine translation.
- *After*: analyze restaurant reviews and rank “must-try” dishes using sentiment analysis.

Accommodation (Sleep)

- *Before*: suggest hotels/homestays within budget using recommendation systems.



- *During*: support Q&A and check-in with NLP hotel chatbot.
- *After*: analyze reviews to predict satisfaction using text mining + sentiment analysis.

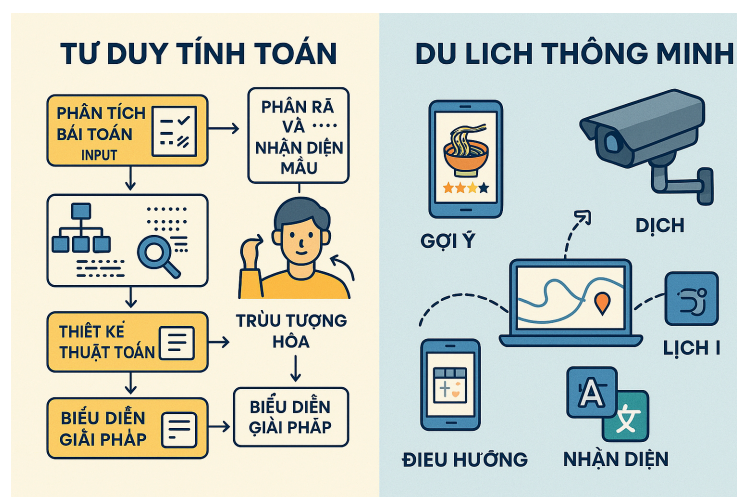
Safety (Safe)

- *During*: detect abnormal weather/disasters with machine learning + open weather data.
- *During*: warn of crowded or risky areas using GPS data; multilingual emergency chatbot using NLP + translation.

3. Example Illustration

Suppose a group selects the topic:

“A system recommending local dishes for tourists during their trip.”



Applying Computational Thinking:

1. Problem Analysis

- Input: user information (preferences: spicy, sweet, vegetarian; budget; GPS location).
- Output: a list of 3–5 suitable restaurants (name, dishes, price, distance).
Example: Given current location, taste, and budget, the system returns a list of restaurants within 3 km.
Use AI models to process complex inputs like “preferences” via natural language understanding.

2. Decomposition & Pattern Recognition

Subdivide the problem into parts:

- Collect restaurant data (name, location, menu, price, rating).
- Filter by distance (≤ 3 km).
- Filter by budget.



- Match taste tags (spicy, vegan, dessert...).
- Rank results (by rating or proximity).
AI can learn user behavior patterns to improve recommendation accuracy.

3. Abstraction

Simplify restaurant data to 4 key attributes:

- Location (GPS),
- Average price,
- Type (tags),
- Rating (score).
Ignore details like phone or photos.
AI can automatically extract these from raw data (e.g., reviews → “cheap–tasty–crowded–clean”).

4. System/Algorithm Design

Design data collection and analysis modules, then a simple recommendation algorithm:

1. Receive user input (location, budget, taste).
2. Filter restaurants within radius R.
3. Filter by budget.
4. Match by taste tags.
5. Sort by rating or distance.
6. Return top 3.
Optionally, replace manual scoring with machine learning to compute a “recommendation score.”

5. Representation

Draw flowcharts or system diagrams showing each process (input → filters → ranking → output).

Include pseudocode and Python libraries, explaining why each is appropriate.

6. Implementation / Simulation

Write a simple Python prototype with 5–10 hardcoded restaurants or real data via Google Maps API.

7. Testing & Reflection

- **Testing:** try different budgets or tastes—does output make sense?
- **Reflection:** note weaknesses—e.g., no open-hours or latest reviews considered.
- **Improvement:** add time-based filters or “weekly trending dishes” from social media data.



Here is a simple introductory suggestion meant to illustrate how to apply computational thinking to a familiar tourism scenario. During the project, each student team should develop its own analysis in greater detail, make each step of the reasoning explicit, and increase the topic's level of complexity. In particular, teams should consider integrating multiple AI models to broaden the scope and enhance practical applicability. In this way, the final product will not only embody logical thinking but also demonstrate the ability to combine modern technologies to solve real-world problems.

4. Suggested Timeline

Weeks	Task
1–2	Problem analysis; define inputs/outputs; refine from ill-defined to well-defined
3–4	Decomposition, pattern recognition, abstraction, system diagram
5	Algorithm design, pseudocode, flowchart
6–7	Small simulation (code, API, Excel/Sheets)
8–9	Testing, feedback, improvement
10–11	Report, presentation, demo

Lectures: instructor explains computational thinking steps, guides group discussions and peer feedback.

Practicals: support with tools, code samples, APIs, and demo technologies.

5. Rubric đánh giá

Criterion	Weight	Excellent (9–10)	Good (7–8)	Fair (5–6)	Poor (<5)
Problem Analysis	10%	Clear input/output, realistic context	Identified but simple	Incomplete	Unclear
Decomposition & Pattern Recognition	10%	Logical, recognizes common patterns	Partial	Fragmented	None
Abstraction	10%	Core elements only; AI-assisted data extraction	Partial	Vague	None
Algorithm Design & Representation	20%	Logical, clear pseudocode/flowchart; includes AI component	Partial	Weak	Not feasible
Implementation / Simulation	25%	Working prototype (Python/Excel/API) with AI integration	Simple mock-up	Weak	None
Testing & Improvement	10%	Multiple test cases, strong analysis, realistic improvement	Basic tests	Minimal	None



Criterion	Weight	Excellent (9–10)	Good (7–8)	Fair (5–6)	Poor (<5)
Report & Presentation	15%	Complete report, clear slides, strong delivery and defense	Weak defense	Unclear	Missing

Performance is assessed based on both **final output** and **process participation**—presentation, discussion, and reasoning. Students must actively explain their thinking and AI usage.

6. Regulations

- Each team: 5–7 students (no fewer).
- Teams register their members and topics; the instructor may adjust in special cases.
- Every member must contribute to all parts.
- Reports must include clear task allocation and progress.
- Weekly progress updates (in class or via LMS).
- Use of ChatGPT or similar tools is allowed for idea generation or code reference, but all content must be understood and rewritten by the team.
- Plagiarism = 0 points.

7. Submission Format

- Report (GroupID.pdf): detailed process, diagrams, pseudocode
- Presentation slides.
- Simulation/demo: working prototype (Python code).
- Team logbook: task division and progress notes.
- Submit via Moodle links provided.

8. Reference Sources

Open Data:

- Google Maps Platform (Places API, Directions API)
- OpenStreetMap
- Tourism data from the Vietnam National Administration of Tourism (VNAT) or provinces

AI & Machine Learning Datasets:

- Kaggle (tourism, food, hotel datasets)
- Hugging Face (NLP models: translation, chatbot)
- GitHub (small recommendation or vision projects)

Academic References:



- Google Scholar (keywords: *Smart Tourism, AI Recommendation in Travel*)
- UNWTO, McKinsey, and World Bank technology trend reports

Design & Simulation Tools:

- Draw.io, Lucidchart (flowcharts, system diagrams)
- Python (pandas, scikit-learn, streamlit for demos)
- Scratch (for visual simulation)

Citation Tools:

- Follow **APA/MLA** or instructor's preferred style.
- Use Zotero, Mendeley, or Google Docs Citation Tool.